Stack / Component	Project need it satisfies	Why this fits
React	Responsive web UI for roster management, start/sit, lineup optimization, waivers, trades, and explanations	Component-based UI fits the many interactive views and must work on desktop + mobile web
TypeScript	Reliable frontend handling of players, leagues, recommendations, API responses, and scoring settings	Strong typing reduces mismatches between complex frontend/backend data structures
FastAPI	API endpoints for recommendations, roster management, platform sync, player data, and authentication	Fast to build, typed, naturally integrates with the Python recommendation/ML layer
Python	Recommendation logic, feature engineering, backtesting, projections, ingestion	The project's core analytical work is statistical/ML-oriented, where Python has the strongest ecosystem
PostgreSQL	Store users, leagues, rosters, players, weekly stats, injuries, recommendations, historical data	Most project data are strongly relational and require reliable querying across seasons/weeks
SQLAlchemy + Alembic	Safely access and evolve the database as requirements change	ORM simplifies backend development; migrations keep schema changes reproducible
pandas / Polars	Transform raw NFL data into usable weekly/player features	Needed for ingestion, cleanup, aggregation, historical analysis, and model input preparation
scikit-learn / gradient boosting	Produce player projections capable of beating the baseline	Directly supports O1 and the start/sit recommendation engine without unnecessary deep-learning complexity

OR-Tools / deterministic optimizer	Generate the highest-value **valid** starting lineup	Projections alone cannot enforce QB/RB/WR/FLEX roster constraints
nflverse	Current and historical NFL player/statistical data	Supplies the raw data required for projections, injuries/context, and backtesting
Sleeper API	Import actual leagues, scoring rules, and rosters	Satisfies the requirement for at least one real fantasy-platform integration
Provider adapter layer	Protect the system from Sleeper/nflverse changes or later provider additions	Necessary because external integrations are explicit project risks
Background worker / scheduler	Weekly data refresh, model updates, injury/status refresh, projection generation	These operations must happen automatically and should not make users wait during a recommendation request
Docker	Consistent development and deployment environment	Backend, worker, dependencies, and ML packages need to behave identically across team machines and production
GitHub Actions	Automated testing and deployment	Supports reliability requirements by preventing broken code or failed integrations from being deployed
Managed hosting	Keep the application available throughout the NFL season	Satisfies O5 without requiring the team to administer production servers manually
Managed PostgreSQL	Persistent, backed-up production storage	Roster, historical, and model data must survive application redeployments/failures

Redis — optional	Cache repeated projections/recommendations if performance becomes an issue	Helps the 15-second latency requirement, but is not actually necessary until PostgreSQL/local caching proves insufficient
Rule/template explanation engine + glossary	Every recommendation must have a 1–2 sentence beginner-friendly explanation with no unexplained jargon	This is a core product requirement, not cosmetic UI; separating it from prediction prevents the explanation layer from changing or inventing recommendations